Smart Contract Security Audit Report

Table Of Contents

1 Executive Summary

On 2024.02.01, the SlowMist security team received the RSS3-Network team's security audit application for RSS3 Layer2, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

The RSS3 Network is consisted of nodes indexing various protocols within the Open Web. The nodes structure open information and ensure it is available and valuable to everyone. The network is powered by the RSS3 token which rewards those that contribute to its formation.

The RSS3 project was developed based on the source code of Optimism, an Ethereum layer 2 project, and the main modifications are contained in the following PRs:

- https://github.com/RSS3-Network/op-geth/pull/1
- https://github.com/RSS3-Network/optimism/pull/2

The main objective of this audit was to review the security impact of source code modifications on the overall project.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Missing the zero address check	Others	Suggestion	Acknowledged
N2	Redundant code	Others	Suggestion	Fixed
N3	Explicitly rejecting ether transfers	Others	Suggestion	Fixed

4 Code Overview

4.1 Contracts Description

Audit scope are all the files below:

https://github.com/RSS3-Network/op-geth/

commit: 205ef451e56c00ab313a39382c68152c11ad8a84

- core/state/statedb.go
- core/vm/instructions.go
- params/protocol_params.go
- rpc/client.go

https://github.com/RSS3-Network/optimism

commit: 0c974379aaefcbd5dcf1ca8cc17637e5338deec6

- packages/contracts-bedrock/src/L1/L1StandardBridge.sol
- packages/contracts-bedrock/src/L1/OptimismPortal.sol
- packages/contracts-bedrock/src/L2/L2StandardBridge.sol
- packages/contracts-bedrock/src/L2/RSS3Token.sol
- packages/contracts-bedrock/src/universal/FeeVault.sol

- packages/contracts-bedrock/src/universal/StandardBridge.sol

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

L1StandardBridge			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	StandardBridge
initialize	Public	Can Modify State	initializer
paused	Public	-	-
depositERC20	External	Can Modify State	onlyEOA
depositERC20To	External	Can Modify State	-
finalizeERC20Withdrawal	External	Can Modify State	-
l2TokenBridge	External	-	-
_initiateERC20Deposit	Internal	Can Modify State	-
_emitERC20BridgeInitiated	Internal	Can Modify State	-
_emitERC20BridgeFinalized	Internal	Can Modify State	-

OptimismPortal			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	Public	Can Modify State	initializer
l2Oracle	Public	-	-

OptimismPortal			
systemConfig	Public	-	-
GUARDIAN	External	-	-
guardian	Public	-	-
paused	Public	-	-
minimumGasLimit	Public	-	-
_resourceConfig	Internal	-	-
proveWithdrawalTransaction	External	Can Modify State	whenNotPaused
finalizeWithdrawalTransaction	External	Can Modify State	whenNotPaused
depositTransaction	Public	Payable	metered
isOutputFinalized	External	-	-
_isFinalizationPeriodElapsed	Internal	-	-

L2StandardBridge			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	StandardBridge
withdraw	External	Payable	onlyEOA
withdrawTo	External	Payable	-
finalizeDeposit	External	Payable	-
l1TokenBridge	External	-	-
_initiateWithdrawal	Internal	Can Modify State	-
_emitERC20BridgeInitiated	Internal	Can Modify State	-
_emitERC20BridgeFinalized	Internal	Can Modify State	-

RSS3Token			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	ERC20

FeeVault			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
<Receive Ether>	External	Payable	-
withdraw	External	Can Modify State	-

StandardBridge			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
messenger	External	-	-
otherBridge	External	-	-
paused	Public	-	-
bridgeERC20	Public	Can Modify State	onlyEOA
bridgeERC20To	Public	Can Modify State	-
finalizeBridgeERC20	Public	Can Modify State	onlyOtherBridge
_initiateBridgeERC20	Internal	Can Modify State	-
_isOptimismMintableERC20	Internal	-	-
_isCorrectTokenPair	Internal	-	-
_emitERC20BridgeInitiated	Internal	Can Modify State	-
_emitERC20BridgeFinalized	Internal	Can Modify State	-

4.3 Vulnerability Summary

[N1] [Suggestion] Missing the zero address check

Category: Others

Content

In the OptimismPortal contract, RSS3Token contract, FeeVault contract, and StandardBridge contract, the zero address check is not performed on the incoming address parameter in the constructor function.

- OptimismPortal.sol#L110-L114

```
    constructor(L2OutputOracle _l2Oracle, SystemConfig _systemConfig) {
        L2_ORACLE = _l2Oracle;
        SYSTEM_CONFIG = _systemConfig;
        initialize(SuperchainConfig(address(0)));
    }
```

- RSS3Token.sol#L12-L3

```
    constructor(address _l2Bridge, address _l1Token) ERC20(_l2Bridge, _l1Token,
"RSS3", "RSS3", 18) { }
}
```

- FeeVault.sol#L51-L55

```
    constructor(address _recipient, uint256 _minWithdrawalAmount, WithdrawalNetwork
_withdrawalNetwork) {
        RECIPIENT = _recipient;
        MIN_WITHDRAWAL_AMOUNT = _minWithdrawalAmount;
        WITHDRAWAL_NETWORK = _withdrawalNetwork;
    }
```

- StandardBridge.sol#L101-L04

```
constructor(address payable _messenger, address payable _otherBridge) {
        MESSENGER = CrossDomainMessenger(_messenger);
        OTHER_BRIDGE = StandardBridge(_otherBridge);
    }
```

Solution

It is recommended to add the 0 address check when sensitive parameters are modified.

Status

Acknowledged

[N2] [Suggestion] Redundant code

Category: Others

Content

- packages/contracts-bedrock/src/L1/OptimismPortal.sol

Two events are declared and not used.

```
event Paused(address account);

event Unpaused(address account);
```

Solution

Delete it if it doesn't work.

Status

Fixed; Fix this in commit: 940d1e3a1abab227057e6e71abae48c82eb1378e

[N3] [Suggestion] Explicitly rejecting ether transfers

Category: Others

Content

In the L1StandardBridge contract, the cross-chain bridge cancels the ETH cross-chain interface, but when the user mistakenly transfers ETH into the contract, there is no operation in the contract to process the ETH.

- L1StandardBridge.sol

Solution

It is recommended to add a receive function in the contract to trigger and revert the transaction when the contract receives ETH.This makes it easy to pinpoint that ETH transfers are not supported.

Status

Fixed; Fix this in commit: 6078c95f7c77843a6f9b3901aa9cdf212010c917

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002402060002	SlowMist Security Team	2024.02.01 - 2024.02.06	Passed

Summary conclusion: The SlowMist security team use a manual and SlowMist team's analysis tool to audit the project, during the audit work we found 3 suggestion vulnerabilities.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.

Official Website

www.slowmist.com

E-mail

team@slowmist.com

Twitter

@SlowMist_Team

Github

https://github.com/slowmist